

Отчёт по прохождению внешнего курса

Этап 3 — Продвинутые темы

Скворцова Анастасия

2026-05-16

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	8
4.1	Раздел 3.1 — Редактор vim	8
4.2	Раздел 3.2 — Оболочки и скрипты	13
4.3	Раздел 3.3 — Условные операторы и циклы	16
4.4	Раздел 3.4 — Арифметика и функции	20
4.5	Раздел 3.5 — Регулярные выражения (grep, sed)	26
4.6	Раздел 3.6 — Построение графиков (gnuplot)	33
4.7	Раздел 3.7 — Права доступа и утилиты	35
5	Выводы	40
6	Список литературы	41

Список иллюстраций

Список таблиц

1 Цель работы

Целью третьего этапа прохождения внешнего курса «Введение в Linux» на платформе Stepik является изучение продвинутых тем: работа с текстовым редактором vim, написание bash-скриптов (переменные, условия, циклы, функции), использование регулярных выражений в grep и sed, построение графиков в gnuplot, управление правами доступа и работа с утилитами wc, du.

2 Задание

Необходимо последовательно изучить материалы курса и выполнить все контрольные мероприятия по разделам третьего этапа:

1. Редактор vim.
2. Оболочки и скрипты (переменные, подстановки, история).
3. Условные операторы и циклы в bash.
4. Арифметика и функции в bash.
5. Регулярные выражения (grep, sed).
6. Построение графиков (gnuplot).
7. Права доступа и утилиты.

По каждому заданию предоставлены скриншоты и пояснения.

3 Теоретическое введение

Третий этап курса посвящён продвинутым инструментам Linux:

- **vim** — мощный текстовый редактор с режимами (нормальный, вставки, командный, визуальный). Выход: `:q`. Режим выделения: `v`.
- **bash** — командная оболочка. Переменные могут содержать буквы, цифры, знак подчёркивания, но не могут начинаться с цифры.
- **Подстановки и история команд**: каждая оболочка имеет свою историю, дочерние оболочки не наследуют историю родительской.
- **Условные операторы**: `if`, `case`, `test` (`[ ]`, `[[ ]]`). Код возврата программы проверяется через `$?`.
- **Циклы**: `while`, `for`. Конструкция `continue` пропускает текущую итерацию.
- **Функции** в `bash`: `gcd()` `{ ... }`. Локальные переменные — `local`.
- **Арифметика**: `let`, `(( ))`, `$(( ))`.
- **Регулярные выражения**: `-E` (extended regex). `grep` для поиска, `sed` для замены.
- **gnuplot**: опция `-persist` для сохранения графиков после закрытия.
- **Права доступа**: `chmod`, `chown`. Владелец, группа, остальные.
- **Утилиты**: `wc` (слова, строки, байты), `du -h -s` (размер директории), `mkdir dir{1..3}` (создание нескольких директорий).

4 Выполнение лабораторной работы

4.1 Раздел 3.1 — Редактор vim

4.1.1 Задание 1

Формулировка и результат прохождения:

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Верно. Так держать!

- ☐ "Esc"
- ☒ ":", затем "q", затем "Enter"
- ☐ "q", затем "Enter"
- ☐ "Ctrl", затем "x"
- ☐ ":", затем "q"

[Следующий шаг](#) [Решить снова](#)

Пояснение: В редакторе vim для выхода из файла (без сохранения) в нормальном режиме нужно нажать :, затем q, затем Enter. Если файл не изменён, команда :q завершает работу. Esc переключает режимы, но не выходит из редактора.

4.1.2 Задание 2

Формулировка и результат прохождения:

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению `w`, нужно открыть vim и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про `word` и `WORD`. Кроме того, можно вызвать сразу справку по термину `word` при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

☒ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Нажимая только на `W`, нельзя переместить курсор на "."
- ☒ В этой строке 9 "слов" (`word`)
- ☒ В этой строке 5 "больших слов" (`WORD`)
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на `W`, чем на `w`
- ☐ В этой строке 12 "слов" (`word`)
- ☐ Чтобы попасть в конец строки, нужно совершить больше нажатий на `W`, чем на `w`

Следующий шаг

Решить снова

Пояснение: В vim существует различие между словом (`word`) и большим словом (`WORD`). `Word` состоит из букв, цифр и подчёркиваний, `WORD` — из любых непробельных символов. Команда `W` перепрыгивает через знаки препинания (например, точку), поэтому для перемещения в конец строки нужно меньше нажатий на `W`, чем на `w`. Я выбрала все верные утверждения.

4.1.3 Задание 3

Формулировка и результат прохождения:

клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☒ ddithree four four four five<Esc>

☐ d2wwywp

☐ x2wwywp

☐ d2dwywPp

☒ d2wwywp

☒ d2wwifour four <Esc>

Следующий шаг

Решить снова

Пояснение: Для преобразования строки «one two three four five» в «three four four four five» можно использовать несколько комбинаций. Подходят варианты d2wwywp и d2wwifour four <Esc>. Остальные варианты либо удаляют не то, либо вставляют в неправильное место.

4.1.4 Задание 4

Формулировка и результат прохождения:

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✓ Правильно.

```
:%s/Windows/Linux
```

Следующий шаг Решить снова

Пояснение: В vim команда `:%s/Windows/Linux` заменяет первое вхождение `Windows` в каждой строке на `Linux`. Символ `%` означает весь файл, `s` — substitute. Без флага `g` заменяется только первое вхождение в строке.

4.1.5 Задание 5

Формулировка и результат прохождения:

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово "Visual". Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Режим выделения открывается из любого другого режима по нажатию "v"
- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись -- VISUAL -- (или -- ВИЗУАЛЬНЫЙ РЕЖИМ --)
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза
- ☒ В режиме выделения можно использовать команды d (удалить) и y (скопировать)
- ☒ Режим выделения открывается из нормального режима по нажатию "v"

Следующий шаг

Решить снова

Пояснение: Режим выделения (Visual mode) открывается из нормального режима клавишей v. Внизу редактора появляется надпись -- VISUAL --. В этом режиме можно перемещаться (w, e, \$), удалять (d) и копировать (y). Выход — дважды Esc. Я отметила все верные утверждения.

4.2 Раздел 3.2 — Оболочки и скрипты

4.2.1 Задание 1

Формулировка и результат прохождения:

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Отличное решение!

- ☐ Из наборов A и C
- ☐ Только из набора A
- ☐ Только из набора B
- ☐ Из наборов B и C
- ☒ Только из набора C

Следующий шаг

Решить снова

Пояснение: Каждая оболочка (`bash`, `sh`) имеет свою собственную историю команд. При запуске дочерней оболочки она не наследует историю родительской. В последней запущенной оболочке будут доступны только команды, введенные в ней самой, то есть набор C.

4.2.2 Задание 2

Формулировка и результат прохождения:

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

☒ Верно.

- ☒ `/home/bi/file1.txt`
- ☐ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)
- ☐ `/home/bi/Documents/file1.txt`
- ☐ `/home/bi/Desktop/file1.txt`

Пояснение: Скрипт меняет текущую директорию на `/home/bi/`, создаёт файл `file1.txt`, затем переходит в `/home/bi/Desktop/`. После завершения скрипта файл остаётся в директории, где был создан, то есть `/home/bi/file1.txt`.

4.2.3 Задание 3

Пояснение: В `bash` имена переменных могут состоять из букв, цифр и знака подчёркивания, но не могут начинаться с цифры. Подходят `variable123` (начинается с буквы). `123variable` начинается с цифры — недопустимо. `var.i.able`, `vari/able`, `vari-i-able`, `var@iable` содержат недопустимые символы.

4.2.4 Задание 4

Формулировка и результат прохождения:

Напишите программу. Тестируется через stdin → stdout

✓ Отличное решение!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 echo "Arguments are:" \ $1=$1 \ $2=$2
2
3
4
5
6
```

Следующий шаг

Решить снова

Пояснение: Скрипт должен выводить строку вида Arguments are: \$1=первый_аргумент \$2=второй_аргумент. Для этого используется echo с экранированием символа \$ перед \$1 и \$2 (чтобы вывести буквально \$1=), а затем подстановка реальных аргументов через \ \$1 (но в решении подруги используется другой синтаксис). В моём варианте команда: echo "Arguments are: \ \$1=\ \$1 \ \$2=\ \$2".

4.3 Раздел 3.3 — Условные операторы и циклы

4.3.1 Задание 1

Формулировка и результат прохождения:

Выберите все подходящие ответы из списка

☒ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ 5 -ge 5
- ☒ -n \$0
- ☒ \$# -ge 0
- ☐ \$# -gt 0
- ☐ -z ""
- ☒ -s \$0

Следующий шаг

Решить снова

Вы получили: 1 балл

Верно решили 33 953 учаш

Пояснение: Условие `$# -ge 0` всегда истинно, так как количество аргументов — неотрицательное целое число. Условие `-z ""` (пустая строка) также всегда истинно. `5 -ge 5` истинно (число 5 больше или равно 5). Остальные условия зависят от контекста и не являются всегда истинными.

4.3.2 Задание 2

Формулировка и результат прохождения:

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

☒ Всё правильно.

- ☐ Сначала one, потом two
- ☐ Сначала two, потом four
- ☒ Сначала four, потом four
- ☐ Сначала two, потом one

Следующий шаг

Решить снова

Пояснение: При `var=3` выполняются сравнения: `3 > 5`? нет, `3 < 3`? нет (строгое), `3 -eq 4`? нет $\rightarrow$ else $\rightarrow$ four. При `var=5`: `5 > 5`? нет (строгое), `5 < 3`? нет, `5 -eq 4`? нет $\rightarrow$ else $\rightarrow$ four. Оба раза выводится four.

4.3.3 Задание 3

Формулировка и результат прохождения:

Напишите программу. Тестируется через stdin → stdout

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1
2 v=student
3 case $1 in
4 0) res="No ${v}s";;
5 1) res="$1 ${v}";;
6 [2-4]) res="$1 ${v}s";;
7 *) res="A lot of ${v}s";;
8 esac
9 echo "$res"
10
11
```

Следующий шаг

Решить снова

Пояснение: Скрипт на bash с использованием case для определения количества студентов. Для 0 — «No students», для 1 — «1 student», для 2-4 — «X students», для остальных — «A lot of students».

4.3.4 Задание 4

Формулировка и результат прохождения:

done

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово **"start"**, а сколько раз слово **"finish"**?

Выберите один вариант из списка

☒ Отлично!

- ☐ 3 раза "start" и 2 раза "finish"
- ☐ 5 раз "start" и 5 раз "finish"
- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 3 раза "start" и 3 раза "finish"

Следующий шаг

Решить снова

Вы получили: 1 балл

Верно решили 36 669 учащихся

Пояснение: В цикле `for str in a , b , c , d` перебираются 5 элементов: `a`, `,`, `b`, `,`, `c`, `d`. Слово `start` выводится для каждого элемента (5 раз). `continue` выполняется, если `$str > "c"` (лексикографически). Условию удовлетворяют элементы `d` (т.к. `d > c`). Таким образом, `finish` выводится 4 раза (кроме итерации с `d`).

4.4 Раздел 3.4 — Арифметика и функции

4.4.1 Задание 1

Формулировка и результат прохождения:

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команд что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, своё решение с другими на [форуме решений](#).

- ☐ let a = a + b
- ☐ let "a+=b"
- ☐ a=\$a+\$b
- ☐ a+=b
- ☒ let "a+=b"

Следующий шаг

Решить снова

Пояснение: Для увеличения переменной a на b можно использовать let "a+=b". Вариант с пробелами let a = a + b неверен (пробелы без кавычек вызывают ошибку). a=\$a+\$b выполнит конкатенацию строк, а не сложение. a+=b не является допустимым синтаксисом.

4.4.2 Задание 2

Формулировка и результат прохождения:

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Правильно.

- ☒ `/home/bi`
- ☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)
- ☐ ``pwd``
- ☐ `pwd`
- ☐ `/home/bi/Documents`

Следующий шаг

Решить снова

Пояснение: Обратные кавычки ``pwd`` выполняют команду `pwd`, и результат подставляется в `echo`. Скрипт меняет директорию на `/home/bi/`, поэтому `pwd` выведет `/home/bi`. `echo` выводит этот результат.

4.4.3 Задание 3

Формулировка и результат прохождения:

двойная (*).

Выберите все подходящие ответы из списка

✓ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ if `program > some\_file.txt`
- ☐ Сначала var=`program`, затем if [[\$var -eq 0]]
- ☐ Ничего сделать нельзя
- ☐ if [[`program` -eq 0]]
- ☒ Сначала запустить program, затем if [[\$? -eq 0]]

Следующий шаг

Решить снова

Пояснение: Если программа выводит что-то в stdout, то в if нельзя использовать её напрямую. Нужно сначала запустить программу, сохранить код возврата в \$?, а затем проверить его условием if [[\$? -eq 0]].

4.4.4 Задание 4

Формулировка и результат прохождения:

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 😊). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст

✓ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг

Решить снова

Пояснение: В задании требовалось найти сумму чисел с использованием глобальной и локальной переменных. Глобальная переменная `s2` накапливает сумму $2 \times (1+2+\dots+10) = 110$. Правильный вывод: `counters are and 110`.

4.4.5 Задание 5

Формулировка и результат прохождения:

```
1 while [ true ]
2 do
3     read n1 n2
4     if [ -z $n1 ]; then
5         echo "bye"
6         break
7     else
8         gcd () {
9             remainder=1
10            if [ $n2 -eq 0 ]
11            then
12                echo "bye"
13            fi
14            while [ $remainder -ne 0 ]
15            do
16                remainder=$((n1%n2))
17                n1=$n2
18                n2=$remainder
19            done
20        }
21        gcd $1 $2
22        echo "GCD is $n1"
23    fi
24 done
25
26
27
```

Пояснение: Скрипт для нахождения НОД двух чисел с использованием алгоритма Евклида. Функция gcd вычисляет наибольший общий делитель, после чего выводится сообщение GCD is X. При пустом вводе выводится bye.

4.4.6 Задание 6

Формулировка и результат прохождения:

```
1 #!/bin/bash
2 while [[ True ]]
3 do
4     read birinchi amal ikkinchi
5     if [[ $birinchi == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ "$birinchi" =~ "^[0-9]+$" && "$ikkinchi" =~ "^[0-9]+$" ]]
10    then
11        echo "error"
12        break
13    else
14        case $amal in
15            "+") let "result = birinchi + ikkinchi";;
16            "-") let "result = birinchi - ikkinchi";;
17            "/" ) let "result = birinchi / ikkinchi";;
18            "*" ) let "result = birinchi * ikkinchi";;
19            "%" ) let "result = birinchi % ikkinchi";;
20            "**" ) let "result = birinchi ** ikkinchi";;
21            *) echo "error" ; break ;;
22        esac
23        echo "$result"
24    fi
25 done
26
```

Пояснение: Скрипт-калькулятор поддерживает операции +, -, *, /, %, **. При вводе exit выводит bye. При неверной операции или нечисловых операндах выводит error.

4.5 Раздел 3.5 — Регулярные выражения (grep, sed)

4.5.1 Задание 1

Формулировка и результат прохождения:

Eddard_Stark_biography.txt.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -n "star*" ?`

Выберите все подходящие ответы из списка

☒ Хорошая работа.

- ☒ Star_Wars.avi
- ☐ Eddard_Stark_biography.txt
- ☒ STARS.txt
- ☐ star_trek_OST.mp3
- ☐ stardust.mpeg

Следующий шаг

Решить снова

Пояснение: `-iname` ищет без учёта регистра, `-name` — с учётом. Команда с `-iname "stark*"` найдёт `Eddard_Stark_biography.txt` (с большой S), а `-name` — нет. `STARS.txt` не содержит `stark`, `Star_Wars.avi` — тоже.

4.5.2 Задание 2

Формулировка и результат прохождения:

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же
- ☒ В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☐ Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным

Следующий шаг

Решить снова

Пояснение: `-name` ищет только имя файла, `-path` — весь путь (включая директорию). В некоторых случаях результат может совпадать. `-name` может найти больше файлов, если нужное имя встречается в разных местах, а `-path` ограничен структурой. Я отметила верные утверждения.

4.5.3 Задание 3

Формулировка и результат прохождения:

Какие(ой) из трех файлов (file1, file2, file3) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*"` ?

Выберите один вариант из списка

☒ Верно. Так держать!

☐ Все кроме file1

☒ Все кроме file3

☐ Все три файла

☐ Только file1

☐ Только file3

Следующий шаг

Решить снова

Пояснение: `-mindepth 2` означает, что поиск начинается с глубины 2 (вложенные директории), `-maxdepth 3` — не глубже 3. Файл file1 (глубина 2) найдётся, file2 (глубина 3) найдётся, file3 (глубина 4) не найдётся.

4.5.4 Задание 4

Формулировка и результат прохождения:

каждой строке есть слово **word**. Если вы выполните на этом файле команды.

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Прекрасный ответ.

- ☒ results.txt будет одинакового размера во всех случаях
- ☐ grep -A 1 "word" file.txt > results.txt и grep -B 1 "word" file.txt > results.txt
- ☐ grep -C 1 "word" file.txt > results.txt
- ☐ grep -A 1 "word" file.txt > results.txt
- ☐ Все, кроме grep "word" file.txt > results.txt

Следующий шаг

Решить снова

Пояснение: Если слово `word` есть в каждой строке из 10 строк, то `grep` с опциями `-A 1`, `-B 1`, `-C 1` из-за границ файла не добавят дополнительных строк (нет соседей за пределами). Все команды выведут 10 строк, поэтому размер `results.txt` будет одинаковым во всех случаях.

4.5.5 Задание 5

Формулировка и результат прохождения:

которые выведет на экран команда `grep -E '[x|X|XL]?[u|]buntu$' text.txt`.

Выберите все подходящие ответы из списка

☒ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сраз своё решение с другими на [форуме решений](#).

- ☒ I prefer Kubuntu
- ☐ Lubuntu is better than Windows
- ☒ Lubuntu is better than Ubuntu
- ☒ Linux is not always Ubuntu
- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☒ Mac OS X, Windows, Ubuntu

Следующий шаг

Решить снова

Вы получили: 2 балла

Верно решили 28 16

Пояснение: Регулярное выражение `[x|X|XL]?[u|]buntu$` ищет строки, заканчивающиеся на `ubuntu`, с возможным префиксом из символов `x`, `X`, `XL`. Подходят строки `Lubuntu is better than Ubuntu`, `Linux is not always Ubuntu`, `Mac OS X, Windows, Ubuntu`. `I prefer Kubuntu` не подходит, так как `K` нет в скобках.

4.5.6 Задание 6

Формулировка и результат прохождения:

Что произойдет, если в команде `sed -n "[a-z]*/p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

☒ Верно.

- ☐ Будут выведены все строки файла text.txt, в которых есть только большие буквы латинского алфавита
- ☒ Каждая строка будет выведена два раза
- ☐ На экран будет выведено всё содержимое файла text.txt
- ☐ На экран ничего не напечатается

Следующий шаг

Решить снова

Вы получили: 1 балл

Верно решили 29 965 учащихся
Из всех попыток 100% верных

Пояснение: Без опции `-n` `sed` печатает каждую строку автоматически. Команда `p` печатает строку ещё раз. В итоге каждая строка, удовлетворяющая шаблону, будет выведена дважды.

4.5.7 Задание 7

Формулировка и результат прохождения:

Внимание! Во время проверки мы не запускаем команду, которую вы ввели на реальном файле с "аббревиатурами" (это небезопасно, можно же ввести `rm -rf /*`)! Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем запускаем её смысловую часть (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах.

К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная "несовместимость" заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. `*`, `+`, `?` и `{}`). Однако эту "несовместимость" легко исправить указав при помощи "(" и ")" какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве "a") нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Напишите текст

✓ Верно.

```
sed -E 's/ ([A-Z][A-Z]+) / abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Пояснение: Команда `sed` для замены аббревиатур (две и более заглавные буквы) на слово `abbreviation` с сохранением пробелов: `sed -E 's/ ([A-Z][A-Z]+) / abbreviation /g' input.txt > edited.txt`.

4.6 Раздел 3.6 — Построение графиков (gnuplot)

4.6.1 Задание 1

Формулировка и результат прохождения:

Вы прошли больше 80% курса, оставьте отзыв

[Оставить отзыв](#) [Нет, спасибо](#)

Вы можете скачать и попробовать применить gnuplot к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске gnuplot, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Хорошие новости, верно!

☒ -p, -persist
☐ Такой опции не существует
☐ -s, -show-plots-after-exit
☐ -raise

Следующий шаг

Решить снова

Пояснение: В gnuplot опция -persist (или -p) позволяет сохранять окна с графиками после закрытия самого gnuplot.

4.6.2 Задание 2

Пояснение: set key autotitle columnhead берёт заголовок из первой строки второго столбца. Первая строка данных не рисуется, поэтому будет нарисовано 9 точек.

4.6.3 Задание 3

Пояснение: Для установки трёх делений на оси X с подписями вида "point 1, value X1" используется конкатенация строк через точку: `set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)`.

4.6.4 Задание 4

Пояснение: Для зеркального отражения графика относительно горизонтали нужно изменить знак Z-координаты ($-x^{**2}-y^{**2}$), для ускорения вращения — изменить шаг (вместо +10 сделать +350 или -10), для обратного вращения — изменить знак приращения.

4.7 Раздел 3.7 — Права доступа и утилиты

4.7.1 Задание 1

Формулировка и результат прохождения:

все верные варианты ответа!

Примечание: запись вида команда1; команда2; команда3 означает, что в терминале последовательно выполнены все три команды (сначала команда1, затем команда2 и, наконец, команда3).

Выберите все подходящие ответы из списка

☒ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ chmod 467 file.txt
- ☐ chmod rwxrw-r-- file.txt
- ☐ chmod u-wx file.txt; chmod g-w file.txt
- ☐ chmod 777 file.txt
- ☒ chmod u+wx file.txt; chmod g+w file.txt
- ☒ chmod ug+w file.txt; chmod u+x file.txt

Следующий шаг

Решить снова

Пояснение: Чтобы установить права rwxrw-r-- (764 в восьмеричной системе), можно использовать `chmod ug+w file.txt; chmod u+x file.txt` (добавить запись владельцу и группе, выполнение владельцу) или `chmod u+wx file.txt; chmod g+w file.txt`.

4.7.2 Задание 2

Формулировка и результат прохождения:

Отдельное спасибо слушателю курса **Alexey Antipovsky** за помощь в оформлении **Примечания 2!**

Выберите все подходящие ответы из списка

✓ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `sudo chmod o+w dir`
- ☒ `sudo chown user:group dir`
- ☐ `chown user:group dir`
- ☐ `sudo chown :group dir`
- ☒ `sudo chmod a+w dir`
- ☒ `sudo chown user dir`

Следующий шаг

Решить снова

Пояснение: Для того чтобы пользователь из группы group мог создать файл в директории dir, созданной root, нужно либо добавить запись для группы (`sudo chmod g+w dir`), либо изменить владельца (`sudo chown user:group dir`), либо добавить запись для всех (`sudo chmod a+w dir`).

4.7.3 Задание 3

Формулировка и результат прохождения:

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Запуск приложений

8 из 8 шагов пройдено
7 из 7 баллов получено

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

☐ Количество предложений

☒ Длину самой длинной строки

☒ Количество строк

☒ Количество слов

☒ Размер файла в байтах

Следующий шаг

Решить снова

Вы получили: 1 балл
[Мои решения](#)

Верно решили 26 076 учащихся
Из всех попыток 23% верных

Пояснение: Команда `wc` (word count) подсчитывает количество строк (-l), слов (-w), байт (-c), символов (-m) и длину самой длинной строки (-L). Количество предложений `wc` не умеет считать.

4.7.4 Задание 4

Формулировка и результат прохождения:

нужно вывести в удобном для чтения формате (например, вместо 2048 байт надо вывести 2.0К) и больше на экран
выводить ничего не нужно). В команде указывайте только необходимые для выполнения задания опции и аргументы,
лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу
в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4М (также на экране может быть выведен еще
и символ ".", обозначающий, что это размер именно текущей директории).

Напишите текст

✓ Верно.

```
du -h -s
```

Следующий шаг

Решить снова

Пояснение: Команда `du -h -s` выводит общий размер текущей директории в удобном для чтения формате. `-h` (human-readable) преобразует байты в К, М, Г; `-s` (summary) показывает только итог без перечисления вложенных файлов.

4.7.5 Задание 5

Формулировка и результат прохождения:

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Хорошие новости, верно!

```
mkdir dir{1..3}
```

Следующий шаг

Решить снова

Пояснение: Кратчайшая команда для создания трёх директорий `dir1`, `dir2`, `dir3` — `mkdir dir{1..3}`. Фигурные скобки раскрываются в `dir1 dir2 dir3`.

5 Выводы

Третий этап курса «Введение в Linux» успешно пройден. Получен сертификат о прохождении курса (приложен отдельным файлом). Освоены следующие темы:

- Работа с текстовым редактором `vim` (режимы, навигация, замена, визуальный режим)
- Написание `bash`-скриптов (переменные, подстановки, условия, циклы, функции)
- Арифметические операции в `bash`
- Регулярные выражения в `grep` и `sed`
- Построение графиков в `gnuplot`
- Управление правами доступа (`chmod`, `chown`)
- Утилиты `wc`, `du`, создание нескольких директорий через `{..}`

Все тестовые и интерактивные задания выполнены с максимальным баллом.

6 Список литературы

1. Введение в Linux — Stepik. URL: <https://stepik.org/course/73>
2. Vim documentation. — URL: <https://vimhelp.org/>
3. GNU Bash Manual. — URL: <https://www.gnu.org/software/bash/manual/>
4. GNU sed documentation. — URL: <https://www.gnu.org/software/sed/manual/>
5. Gnuplot documentation. — URL: <http://www.gnuplot.info/documentation.html>
6. Регулярные выражения. — URL: <https://www.gnu.org/software/grep/manual/>